Status-LEDs

Lernunterlage: Online, offline, unbekannt – vom Tuya-Feld bis zur leuchtenden LED

1 Was man sieht

- **Status-Leiste oben im Dashboard:** eine große LED mit einem Satz („Alles in Ordnung“, „1 Gerät ist offline“, „Server nicht erreichbar“), daneben LEDs für Server, Tuya-Cloud und Blink und die Zahlen online / offline / ohne Status. Ein Knopf „Aktualisieren“ fragt sofort neu.
- **Raumkarten:** vor jedem Gerätenamen eine kleine LED.
- **Geräte-Kacheln:** oben rechts eine LED mit Text „online“ oder „offline“.

Farbe	Bedeutung
grün (pulsiert sanft)	online: Tuya meldet das Gerät als verbunden bzw. Dienst erreichbar
rot	offline: Tuya meldet das Gerät als nicht verbunden bzw. Dienst nicht erreichbar
grau	unbekannt: nicht mit Tuya verknüpft, Tuya meldet nichts, Tuya nicht eingerichtet/erreichbar
gelb (pulsiert)	Achtung: z. B. Geräte offline, Tuya-Cloud weg oder Blink braucht eine neue Anmeldung

2 Woher kommt „online“?

Die Tuya-Geräteliste (`GET /v1.0/iot-01/associated-users/devices`) enthält pro Gerät ein Feld `online`. Unsere Geräte kennen ihr Tuya-Gerät über `TuyaDeviceId`. Der neue Endpunkt `GET /api/status` verbindet beides.

```
Datenbank (Devices)          Tuya-Cloud (Geräteliste)          Ergebnis
Lampe   → tuya-abc   ──►     tuya-abc  online: true     ──►    online   (grün)
Stecker → tuya-def   ──►     tuya-def  online: false    ──►    offline  (rot)
Sensor  → (keins)                                      ──►    unknown  (grau)
```

Die Entscheidung trifft `DeviceOnlineEvaluator.Evaluate`, eine **reine Funktion**: Sie bekommt Daten und gibt Daten zurück, ohne Datenbank und ohne HTTP. Deshalb lässt sie sich mit wenigen Zeilen testen.

3 Zwischenspeicher (Cache) im Backend

Die Oberfläche fragt jede Minute nach, auf jedem Gerät, das die App offen hat. Ohne Cache wäre jede Frage ein Aufruf bei der Tuya-Cloud, und das Cloud-Projekt hat ein begrenztes Kontingent.

```
public async Task<IReadOnlyDictionary<string, bool?>> GetAsync(Func<…> load, CancellationToken ct)
{
    await gate.WaitAsync(ct);                    // nur einer gleichzeitig
    try
    {
        if (cached is not null && now - cachedAtUtc < MaxAge)   // jünger als 60 s?
            return cached;                        // → gespeicherte Antwort
        var devices = await load(ct);             // sonst Tuya fragen
        cached = …; cachedAtUtc = now;            // und merken
        return cached;
    }
    finally { gate.Release(); }
}
```

- **Singleton:** Der Cache muss zwischen den Anfragen erhalten bleiben. Den TuyaClient (kurzlebig) bekommt er deshalb als Funktion beim Aufruf, nicht im Konstruktor.
- **SemaphoreSlim:** Kommen zwei Anfragen gleichzeitig, fragt nur die erste bei Tuya, die zweite bekommt danach das gespeicherte Ergebnis.
- **Fehler werden nicht gespeichert:** Ist Tuya kurz weg, versucht es der nächste Aufruf erneut.
- **TimeProvider:** Im Test wird die Uhr vorgestellt (59 s: kein neuer Aufruf, 60 s: neuer Aufruf).

> **Immer 200:** /api/status meldet „Tuya nicht erreichbar“ als Inhalt (tuya: offline), nicht als Fehlercode. Für die Oberfläche ist das eine rote LED, kein kaputter Endpunkt.

4 Frontend: ein Provider, viele LEDs

Datei	Aufgabe
api/status.ts	Typen (LedState, OnlineState, SystemStatus) und getStatus(), getBlinkStatus()
status/StatusProvider.tsx	Fragt alle 60 s und beim Zurückkehren in die App (visibilitychange); Context für alle
status/statusContext.ts	useStatus(): status, blink, serverUnreachable, refresh(), stateOf(id)
components/StatusLed.tsx	Die LED selbst: farbiger Punkt, optional mit Text
components/StatusBar.tsx	Leiste oben im Dashboard mit Gesamtzustand

- **Context statt Prop-Drilling:** Status-Leiste, Raumkarten und Kacheln brauchen dieselben Daten. Ein Provider fragt einmal, alle lesen mit `useStatus()`. Dasselbe Muster wie `useAuth()`.
- **Aufräumen im useEffect:** clearInterval und removeEventListener in der Cleanup-Funktion. Sonst liefen nach dem Abmelden Abfragen weiter.
- **useMemo:** Die Nachschlagetabelle Geräte-Id → Zustand wird nur bei einer neuen Antwort gebaut.
- **Blink seltener:** nur beim Start und auf Knopfdruck, nicht jede Minute (keine offizielle API, Sperrrisiko).

Lint-Warnung: setState direkt im Effekt

oxlint meldete `react(set-state-in-effect)`. Der Effekt rief `loadStatus()` auf, und die Funktion setzte State. Der erste Versuch, nur `setRefreshing(true)` zu entfernen, reichte nicht: Bei einer `async`-Funktion erkennt der Linter nicht sicher, dass das setState erst nach dem `await` passiert. Erst das Muster aus der React-Doku hat geholfen: State nur in `.then()` bzw. `.catch()` setzen.

```
// vorher (Warnung): async-Funktion mit setState
const loadStatus = useCallback(async () => {
  setRefreshing(true)                       // synchron → sofort neuer Render
  try { setStatus(await getStatus()) } ...
}, [])

// nachher (0 Warnungen): State erst, wenn die Antwort da ist
const loadStatus = useCallback(() =>
  getStatus()
    .then((next) => { setStatus(next); setServerUnreachable(false) })
    .catch(() => setServerUnreachable(true))
    .finally(() => setRefreshing(false)), [])

const refresh = useCallback(() => {
  setRefreshing(true)                       // Klick = Ereignis, hier ist setState richtig
  loadStatus(); loadBlink()
}, [loadStatus, loadBlink])
```

Merksatz: Effekte synchronisieren mit der Außenwelt (Server, Timer). State aus einer Benutzeraktion gehört in den Event-Handler, State aus einer Server-Antwort in den Promise-Callback. Und: Erst prüfen (Linter erneut laufen lassen), dann „behoben“ sagen.

Nicht erreichbar oder Fehlerantwort?

Im ersten Test zeigte die Leiste „Server nicht erreichbar“, obwohl Räume und Geräte luden. Grund: Es lief noch ein älteres Backend ohne `/api/status`. Es antwortete mit 404, und jede Fehlerantwort wurde als „nicht erreichbar“ gedeutet. Jetzt unterscheidet `ApiError` (in api/http.ts) beides:

Fall	Erkennung	Anzeige
Keine Antwort (Netzwerk, Backend aus, Proxy ohne Backend)	fetch wirft, oder Fehler ohne ProblemDetails → unreachable	rot: „Server nicht erreichbar“
Backend antwortet mit Fehler (404, 500 mit ProblemDetails)	ApiError mit status, unreachable = false	gelb: „Status nicht abrufbar“ + Hinweis

Merksatz: Eine Fehlermeldung muss die Ursache treffen. „Server nicht erreichbar“ bei einer 404 schickt einen auf die falsche Fährte. Deshalb Fehler nach Art unterscheiden (Status-Code, wer hat geantwortet).

5 CSS: die LED leuchtet

```
.led { --led: var(--offline); width: 10px; height: 10px; border-radius: 50%;
       background: var(--led);
       box-shadow: 0 0 0 3px color-mix(in srgb, var(--led) 18%, transparent),
                   0 0 10px color-mix(in srgb, var(--led) 70%, transparent); }
.led-online  { --led: var(--online); }
.led-offline { --led: var(--danger); }
@media (prefers-reduced-motion: no-preference) {
  .led-online { animation: led-pulse 2.6s ease-in-out infinite; }
}
```

- **Eine Variable pro LED** (`--led`): Jede Farbvariante setzt nur die Farbe, Größe und Leuchten bleiben gleich.
- **color-mix()** erzeugt den halbdurchsichtigen Lichthof aus derselben Farbe.
- **Barrierefreiheit:** Farbe allein reicht nicht (Rot-Grün-Schwäche). Jede LED hat Text (sichtbar oder als aria-label), und das Pulsieren entfällt bei „Bewegung reduzieren“.

6 Kurzantworten für Rückfragen

Frage	Antwort
Woher weiß die App, ob ein Gerät online ist?	Aus dem Feld online der Tuya-Geräteliste, verbunden über TuyaDeviceId.
Belastet das die Tuya-Cloud?	Höchstens ein Aufruf pro Minute dank 60-Sekunden-Cache im Backend.
Warum grau statt rot bei fehlenden Daten?	Rot hieße „sicher offline“. Ohne Daten wissen wir es nicht, also grau.
Warum ein Context?	Mehrere Komponenten brauchen dieselben Daten. Ein Provider fragt einmal, alle lesen mit.
Kann der KI-Agent das nutzen?	Ja, GET /api/status mit dem API-Schlüssel: „Welche Geräte sind offline?“

7 Merksätze

- Reine Funktion = leicht testbar (Daten rein, Daten raus).
- Cache vor teuren oder begrenzten Aufrufen, Fehler nicht cachen.
- Gemeinsame Daten → Context + Provider, Abfragen im useEffect mit Aufräumen.
- Unbekannt ist nicht offline: grau statt rot.
- Farbe nie allein: immer auch Text, Bewegung nur wenn erlaubt.